

Backend Team Document

Context Compression Engine — Backend Build Guide

Your role: You own everything that happens after the frontend sends raw text, up until a clean JSON response goes back. This includes the compression pipeline logic, all Claude API orchestration, and wiring to the database.

1. Your Responsibilities

- Build the two API routes: `/api/compress` and `/api/validate`, plus `/api/history`
- Implement Stage 1 (structural) compression logic
- Orchestrate Stage 2 (semantic) compression via Claude API calls
- Implement the accuracy-validation Q&A + similarity scoring logic
- Handle token counting, cost estimation, latency measurement
- Connect to the database (write run logs, read history)
- Handle errors/timeouts gracefully with fallbacks

2. Tech Stack You'll Use

- Node.js (via Next.js API routes, `app/api/.../route.ts`)
- Anthropic SDK (`@anthropic-ai/sdk`) for Claude API calls
- A tokenizer library (e.g. `gpt-tokenizer` or `tiktoken` — approximate is fine, Claude doesn't expose exact tokenizer publicly, so use an estimate: `~4 chars = 1 token`, or use a JS tokenizer package for consistency)
- Supabase JS client (`@supabase/supabase-js`) for database calls

3. Step-by-Step Build Plan

Step 1: Project Setup (30 min)

1. Set up environment variables: `ANTHROPIC_API_KEY` , `SUPABASE_URL` ,
`SUPABASE_SERVICE_KEY`
2. Install dependencies: `npm install @anthropic-ai/sdk @supabase/supabase-js gpt-tokenizer`
3. Create folder structure:

```
/app/api/compress/route.ts
/app/api/validate/route.ts
/app/api/history/route.ts
/lib/compression/stage1.ts
/lib/compression/stage2.ts
/lib/compression/chunker.ts
/lib/validation/qa.ts
/lib/validation/similarity.ts
/lib/tokenCount.ts
/lib/claudeClient.ts
/lib/supabaseClient.ts
```

Step 2: Utility Setup (30 min)

1. `lib/claudeClient.ts` — export a configured Anthropic client instance (reads API key from env, never hardcode)
2. `lib/tokenCount.ts` — export a function `countTokens(text: string): number` using your chosen tokenizer package
3. `lib/supabaseClient.ts` — export a configured Supabase client instance

Step 3: Stage 1 — Structural Compression (1.5–2 hrs)

Build `lib/compression/stage1.ts` . This is pure logic, **no LLM calls** — must be fast.

Implement these functions and chain them in order:

1. `removeExactDuplicateLines(text)` — split by line, dedupe consecutive/repeated identical lines (keep first occurrence), rejoin
2. `collapseWhitespace(text)` — remove excessive blank lines (more than 1 consecutive), trim trailing whitespace per line
3. `stripRepeatedBoilerplate(text)` — detect patterns like repeated log headers/timestamps or repeated import blocks using regex or line-frequency analysis; if a line/block appears more than N times, keep only the first instance and note a marker like `[boilerplate collapsed x12]`

4. `collapseNearDuplicateLines(text)` — for lines with high string similarity (use a simple similarity function — Levenshtein or Jaccard on tokens — threshold e.g. >85% similar), keep the first, drop the rest
5. Combine all four into one exported function: `runStage1(text): { compressedText, originalTokens, stage1Tokens }`

Test this in isolation first with a sample log file before moving to Stage 2.

Step 4: Stage 2 — Semantic Compression (3–4 hrs)

Build `lib/compression/chunker.ts`:

- `chunkText(text): Chunk[]` — split Stage-1 output into logical chunks. Simple approach: split by double newline (paragraphs) or by function/block boundaries for code (split on top-level function definitions using regex), cap each chunk at ~500 tokens
- Each `Chunk = { id, text, tokenCount }`

Build `lib/compression/stage2.ts`:

1. `scoreChunks(chunks: Chunk[]): ScoredChunk[]`
 - Batch chunks into a **single Claude API call** (don't call once per chunk — too slow/costly)
 - Prompt Claude with all chunk texts labeled by ID, ask it to return a JSON array of `{ id, score (1-10), reason }` rating information density/relevance
 - Use the structured-JSON-output approach: instruct Claude to return ONLY valid JSON, parse it, handle parse failures gracefully (retry once, then fallback to treating all chunks as score 5)
2. `compressLowScoreChunks(scoredChunks): CompressedChunk[]`
 - For chunks scoring below a threshold (e.g. <6), send them to Claude with a prompt like: "Summarize this content, preserving all entities, numbers, variable names, and key facts. Be as concise as possible." — batch multiple low-score chunks into one call where possible
 - For chunks scoring at/above threshold, keep verbatim (no compression, no LLM call needed — saves cost/time)
3. `runStage2(chunks): { compressedText, finalTokens }` — reassemble all chunks (compressed + verbatim) in original order

Step 5: `/api/compress` Route (1 hr)

1. Accept `POST` with body `{ originalText: string }`
2. Validate input (non-empty, under length cap — reject/truncate if >50,000 chars, return clear error)
3. Run `runStage1(originalText)`
4. Run `runStage2(stage1Output)`
5. Compute `compressionRatio = (originalTokens - finalTokens) / originalTokens * 100`
6. Return JSON: `{ compressedText, stage1Text, originalTokens, stage1Tokens, finalTokens, compressionRatio }`
7. Wrap the whole handler in `try/catch` — on Claude API failure, fall back to returning Stage-1-only result with a `warning` field explaining semantic compression was skipped

Step 6: `/api/validate` Route (1.5 hrs)

Build `lib/validation/qa.ts`:

1. `askClaude(context: string, question: string): { answer, latencyMs }` — single function that sends context+question to Claude, measures response time
2. In the route: accept `POST` with `{ originalText, compressedText, question }`
3. If no question provided, auto-generate one: send `originalText` to Claude first asking “Generate one specific factual question that can only be answered correctly using this content” — use that as the question
4. Call `askClaude(originalText, question) → Answer A + latency A`
5. Call `askClaude(compressedText, question) → Answer B + latency B`
6. Build `lib/validation/similarity.ts`:
 - `scoreSimilarity(answerA, answerB): number` — easiest reliable approach for a hackathon: send both answers to Claude and ask it to rate semantic similarity 0-100 with brief reasoning; parse the number out of the JSON response
7. Return JSON: `{ answerFull, answerCompressed, similarityScore, latencyFull, latencyCompressed }`

Step 7: Database Logging (1 hr)

1. After `/api/compress` and `/api/validate` both complete (or trigger from frontend after both calls finish), write a row to the `runs` table (see Database doc for schema) via Supabase client
2. Build `/api/history` route: `GET` → query the `runs` table, return the last 20 runs ordered by timestamp descending

Step 8: Error Handling & Fallbacks (ongoing, budget 1 hr at the end)

1. Every Claude API call must have a timeout (e.g. 20s) and a catch block
2. If chunk scoring fails → treat all chunks as medium priority, apply light compression to all
3. If Q&A validation fails → return partial results with an error flag, don't crash the whole response
4. Log errors to console with enough context to debug quickly during the demo

4. What "Done" Looks Like

- `/api/compress` reliably takes raw text and returns compressed text + accurate metrics in under 20 seconds for a 5,000-token input
- `/api/validate` reliably returns two answers + a similarity score
- Both routes handle a bad/empty input without crashing
- History is retrievable via `/api/history`

5. Coordination Notes

- Agree on exact JSON field names with Frontend team **before** building (use the contracts above as the source of truth — don't rename fields later)
- Get the `runs` table schema finalized with Database team before Step 7
- Give Frontend a mock/stub response early (hardcoded JSON matching the contract) so they can build UI in parallel without waiting on your pipeline